

Phase I

(Setup)

External libraries:-

python-can (import can)
pyisotp (import isotp)
openpyxl (import openpyxl)

In-build libraries:-

time (import time)
threading (import threading)
re (import re)

T
O
O
L

C
O
D
E

Define (core/can_interface.py):
[init_hardware_bus()]



Call: init_hardware_bus()
Config: interface='udp_multicast'
Params: channel=MULTICAST_IP,
port=MULTICAST_PORT ('224.0.0.1' /
5001)



Call: create_physical_stack(bus,
TARGET_TX_ID, TARGET_RX_ID)
[core/transport_layer.py] Setup:
isotp.Address(rxid=0x7E8, txid=0x7E0)
Params: ISOTP_PARAMS stmin=10,
blocksize=8, tx_padding=0x00



Call: create_functional_stack(bus,
TARGET_RX_ID)
Setup: isotp.Address(rxid=0x7E8,
txid=FUNCTIONAL_ID=0x7DF)
dedicated broadcast channel, reserved
for Tester Present frames



Call: engine = DiagnosticEngine(
physical_stack) [core/diagnostic_engine.py]
Call: translator = Translator() loads
config/decode_values.xlsx ONCE at init
[core/translator.py]
Call: tester_present = TesterPresentManager(
engine) idle until a Programming / Extended
session starts



continues in Phase II
(User Request Generation)

E
C
U

C
O
D
E

Define:
[setup_can_bus()]
[fetch_mock_ecu_state()]
[monitor_user_input()]



Call: setup_can_bus() Config:
interface='udp_multicast' Params:
channel='224.0.0.1', port=5001



Call: stack_physical = isotp.CanStack()
Setup: rxid=0x7E0, txid=0x7E8 Params:
stmin=10, blocksize=8, tx_padding=0x0



Call: stack_functional = isotp.CanStack()
Setup: rxid=0x7DF (functional),
txid=0x7E8 NEW: ECU now also listens on
the broadcast 0x7DF channel, in addition
to the physical 0x7E0 channel



Call: threading.Thread(
target=monitor_user_input,
daemon=True).start()
background key listener: 'p' = force Positive, 'n'
= force randomized-NRC fault mode
EV values are now read LIVE, per request,
directly from config/ev_state_final.xls



continues in Phase III
(The ECU Interpretation)

PHASE II

(User request generation)

(Tool code)

GUI (gui_main.py): command_input.text() captured on
Send-button click
eg. typed: "22 02 01"

Build & Validate Request Frame

Call: `raw_text.replace(" ", "")`
Call: `bytes.fromhex(sanitized_input)`
eg output: `b'\x22\x02\x01'`

Extract Target ID

```
if sid == TWO_BYTE_DID_sid and len(payload) >= 3:  
    target_id = (payload[1] << 8) | payload[2] (2-byte DID)  
elif len(payload) >= 2:  
    target_id = payload[1]  
elif target_id = 0x00
```

Session Management (only on positive SID 0x10)

subfunc 0x02 (Programming) / 0x03 (Extended) `tester_present.start()`
background thread sends bytes([0x3E,0x80]) over the FUNCTIONAL
stack every 2 s subfunc 0x01 (Default) `tester_present.stop()`

Execution Call:

GUI: spawns a `UDSWorker(QThread)` that calls `engine.send(payload)`
OFF the GUI thread the window never freezes while waiting on the
bus
Internal Sub-Call: `stack.send(payload)` [core/diagnostic_engine.py]

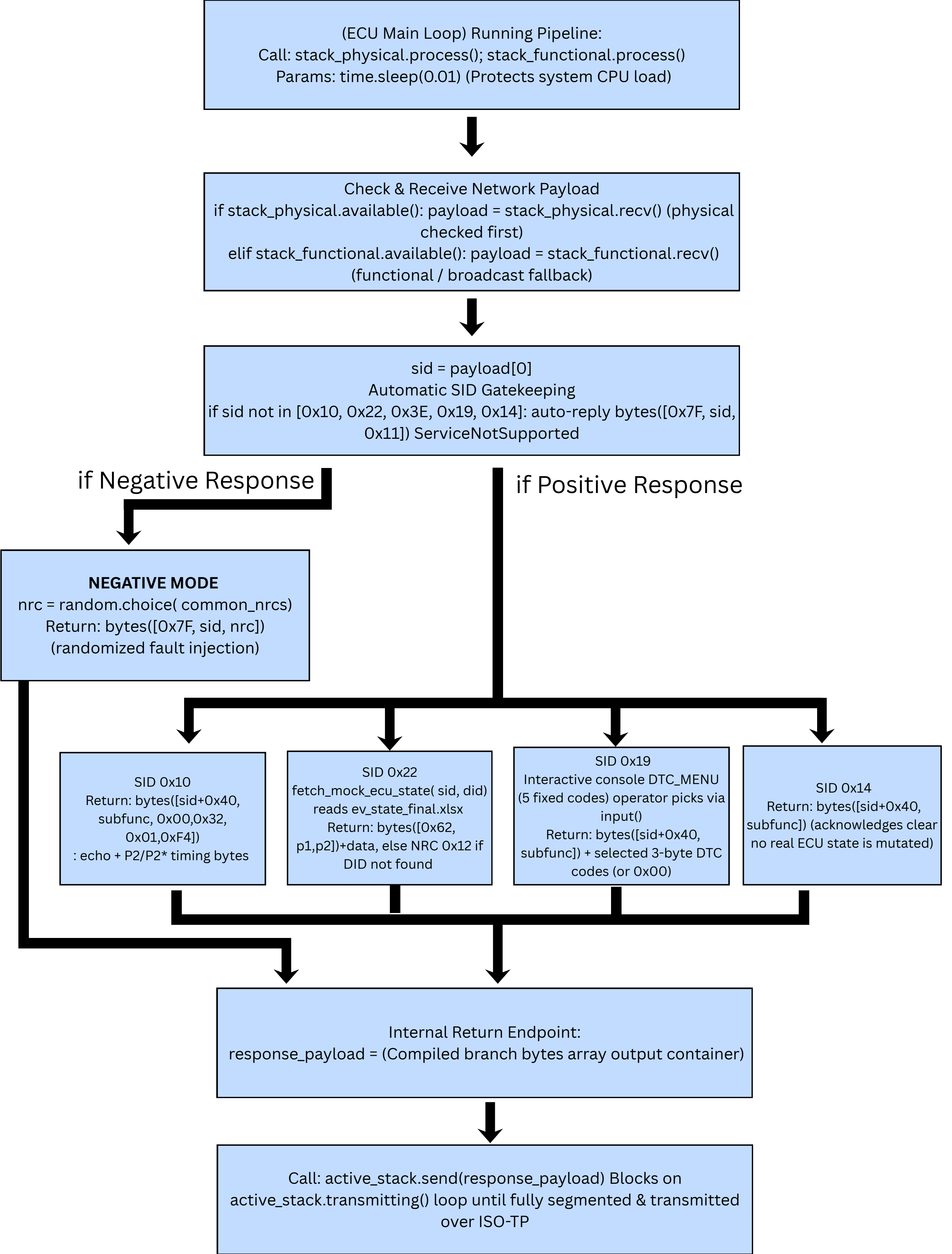
TX PIPELINE: ISO-TP Slices
Multi-frame Data Packets

continues in Phase III
(The ECU Interpretation)

Phase III

(The ECU Interpretation)

(ECU code)



PHASE IV

(Decoding and Showing the result)

(Tool code)

Call: engine.receive(DEFAULT_TIMEOUT=15s)
[core/diagnostic_engine.py]
Loops stack.process() + stack.available() until data arrives or
the
15 s window elapses
eg returns: b'\x62\x02\x01\x55'

Call: evaluate_response(sid, payload, response)
[core/uds_layer.py]
if response is None "Diagnostic Session Timeout"
elif response[0]==0x7F decode_nrc_code(nrc) looked up in a
local
22-entry NRC dictionary (NEGATIVE)
elif response[0]==sid+0x40 POSITIVE; slice payload:
sid==0x22 data[3:], sid==0x14 data[4:], else data[1:]

(If response data byte
starts with 0x7F)

(If response indicates a
positive completion flag)

NEGATIVE PATH

CLI: print(message)
GUI: log_display.append(
message)
eg: "[-] Negative Response
Code (NRC):ConditionsNotCorrect"

POSITIVE PATH

Call: translator.decode(sid,
target_id, data) [core/translator.py]
if sid==0x19: return
data.hex().upper() (raw hex
no longer a formatted DTC list)
else: regex-match DID against
PRELOADED decode_values.xlsx
→ scale_value():
value = raw*factor + offset
→ "Name: value unit"
else (no match): data.hex().upper()

PRESENTATION

CLI: print(f">> {decoded_value}") to console
GUI: handle_worker_result() slot runs on the GUI thread
(triggered by the UDSWorker's pyqtSignal) appends to
log_display and updates session_label / tester_present_label

